

گزارشکار پروژه 3 هوش مصنوعی

دنیز مصطفی زادگان / 810102603

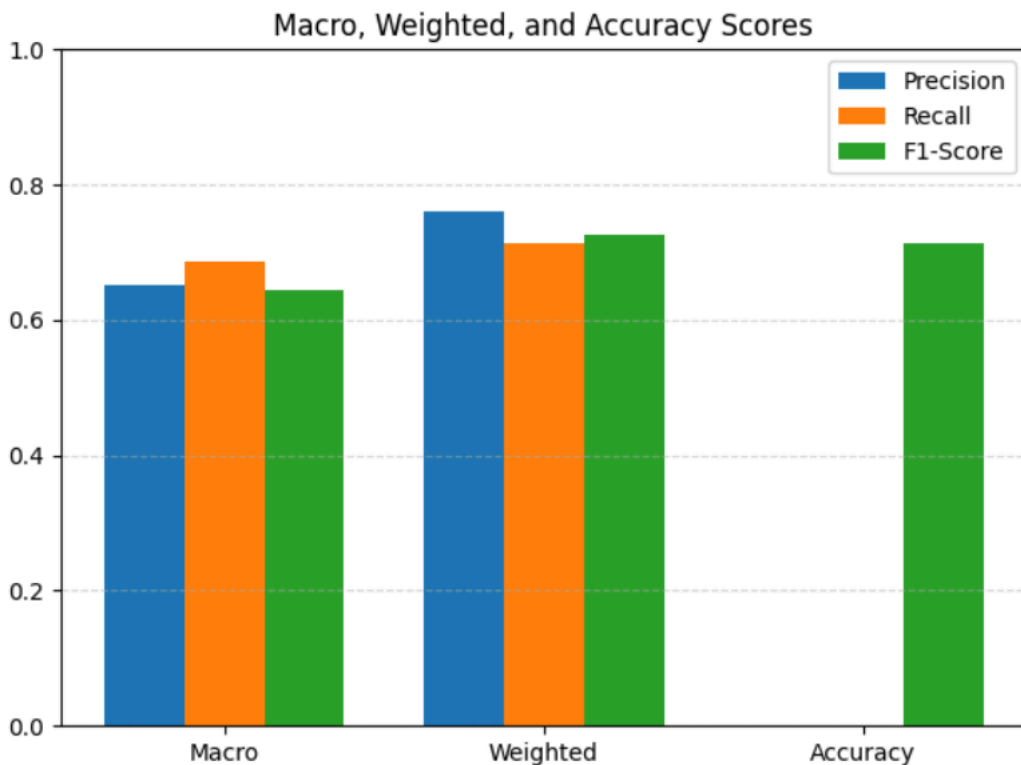
پیشپردازش دادگان :

- **Absences** : چون دارای مقادیر زیاد با پراکندگی زیاد و پرت دارد برای آن از لگاریتم استفاده کردیم تا توزیع آن متعادل تر بشود .
- **Age , eps grade , ds grade** : چون دامنه محدودی داشت ولی مقادیر بسیار بزرگ تر از بقیه داده ها برای اینکه مدل دچار اشتباه نشه برای بزرگی این اعداد آن هارا بین 0 و 1 اسکیل کردیم .
- **Final grades** : رو هم با توجه به صورت پروژه دسته بندی کردیم .
- **motherJob و fatherJob و reason** : این feature هارو حذف کردم چون مقدار داده های گمشده ان ها (other) زیاده و کمکی به مدل ما نمیکنه و اگر اون هارو با یک مقدار دیگه جایگذاری کنیم چون تعدادشون بالاست ممکنه باعث خطا بشه .
- **'sex', 'address', 'universitySupport', 'paid', 'higher', 'internet', 'romantic'** : رو از حالت دسته بندی به عدد با روش labeling تبدیل کردیم تا مدل ها بتوانند با این ویژگی ها کار کنن
- **DSGrad و EPSGrade** : با توجه به نتیجه corr این داده ها روی تصمیم گیری نهایی تاثیر دارند برای همین ان ها را نگه میداریم و Normaliz میکنیم . چون نمره ها محدوده مشخص دارند از روش Min-Max استفاده میکنیم برای نورمال کردن .

Naive Bayes

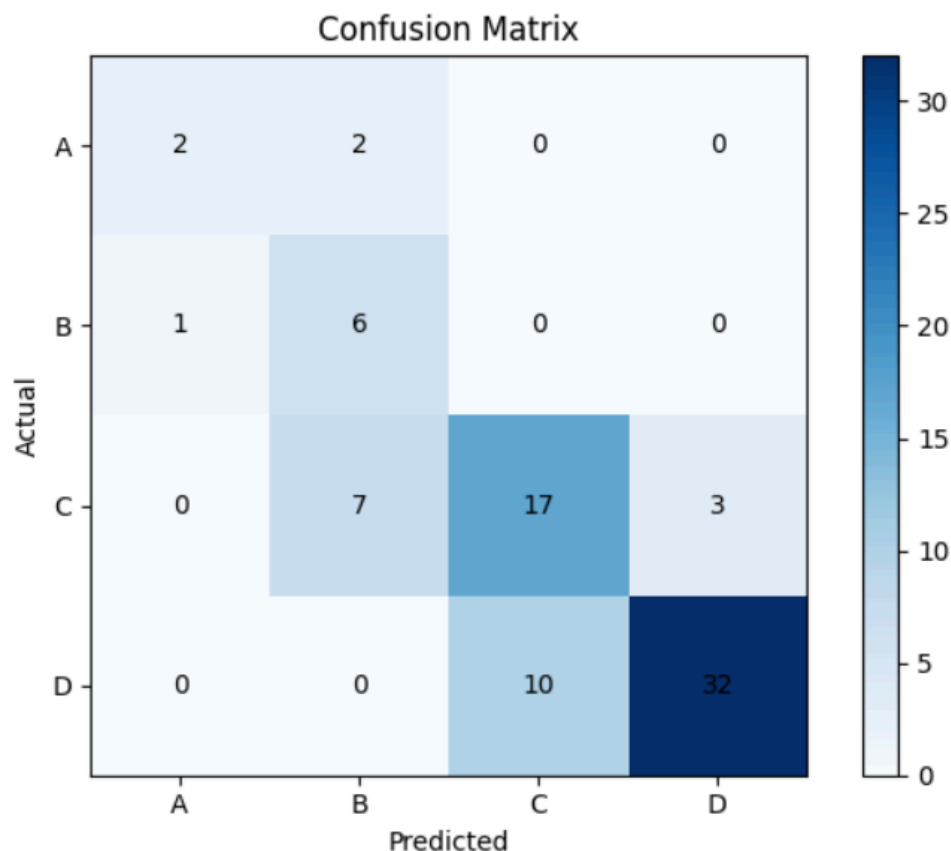
تحلیل :

- Recall : از چیزهایی که واقعاً مثبت بودن، چندتاشو مدل درست گفته؟
- Precision : از چیزهایی که مدل گفته مثبت هستن، چندتاش واقعاً مثبت بودن؟
- F1-Score : حد بین آن دو هست .
- Accuracy : تقسیم کل پیش بینی های درست به کل .
- Weighted : به کلاس هایی با جمعیت بیشتر وزن بالاتری میدهد .
- Macro : وزن همه کلاس ها یکسان هستند .



در نمودار اگر Precision و Recall پایین باشند یعنی مدل دقت پایینی داشته که چون در weighted وزن کلاس های بیشتر بالاتر هست . اگر نموداری در آن مقدارش بیشتر باشد یعنی مدل در کلاس هایی که داده بیشتری دارد بهتر عمل میکند . و برای مقایسه نهایی F1-Score مهم تر هست چون در نهایت برای دقت بالا تعادل بین آن دو اهمیت دارد .

- ماتریس درهم ریختگی :



در این ماتریس هرچه اعداد روی قطر بزرگ تر باشند یعنی مدل بهتر عمل کرده . کلاس D دقت بیشتری داشته چون عدد روی قطرش بیشتره و A دقت کمتر چون عدد خارج قطر بیشتر داره .

یک کلاس Recall = ردیف مربوط به اون کلاس

یک کلاس Precision = ستون مربوط به اون کلاس

Decision Tree

تحلیل :

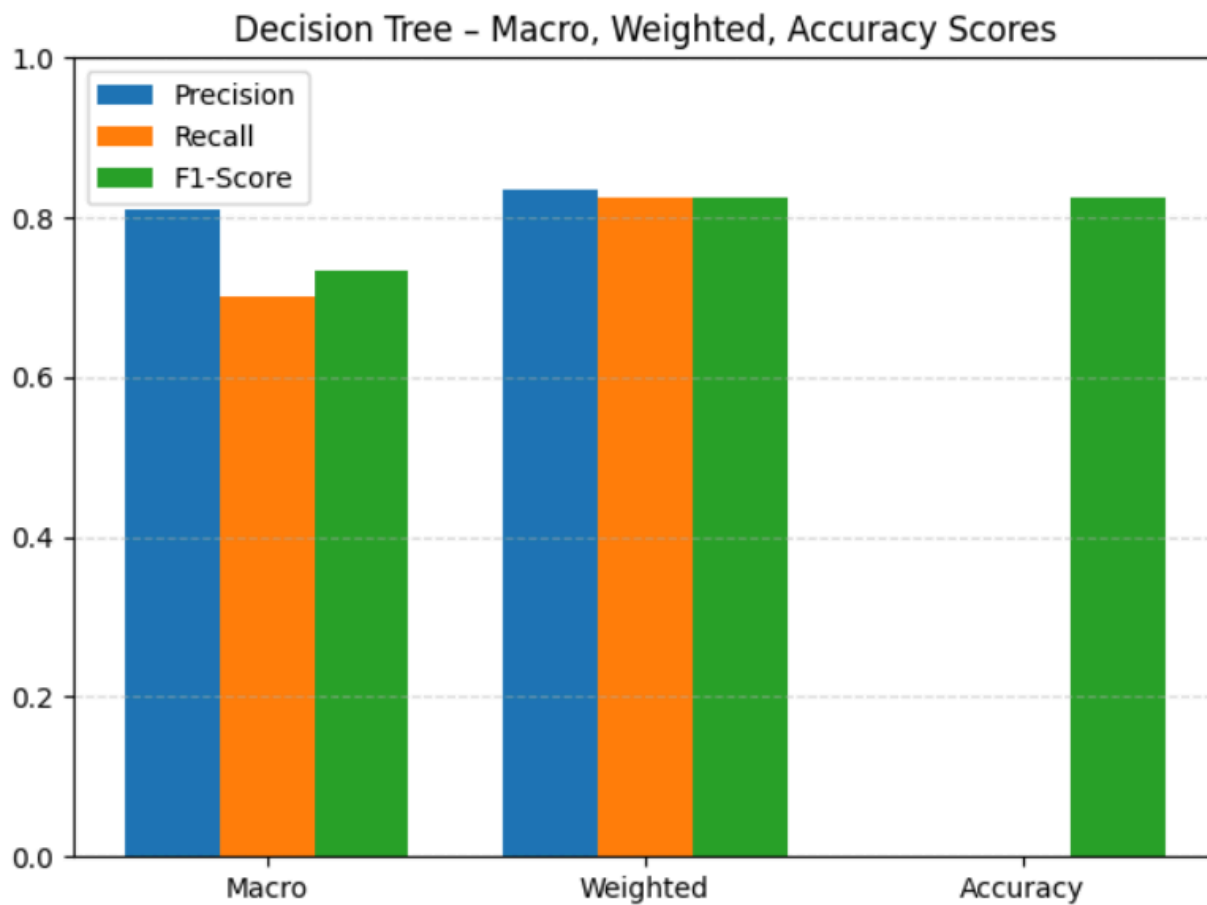
در ابتدا مدل را اجرا کردیم و $Accuracy = 0.7$ و درخت رسم شده بسیار شلوغ شد سپس برای بهینه کردن مدل سه پارامتر

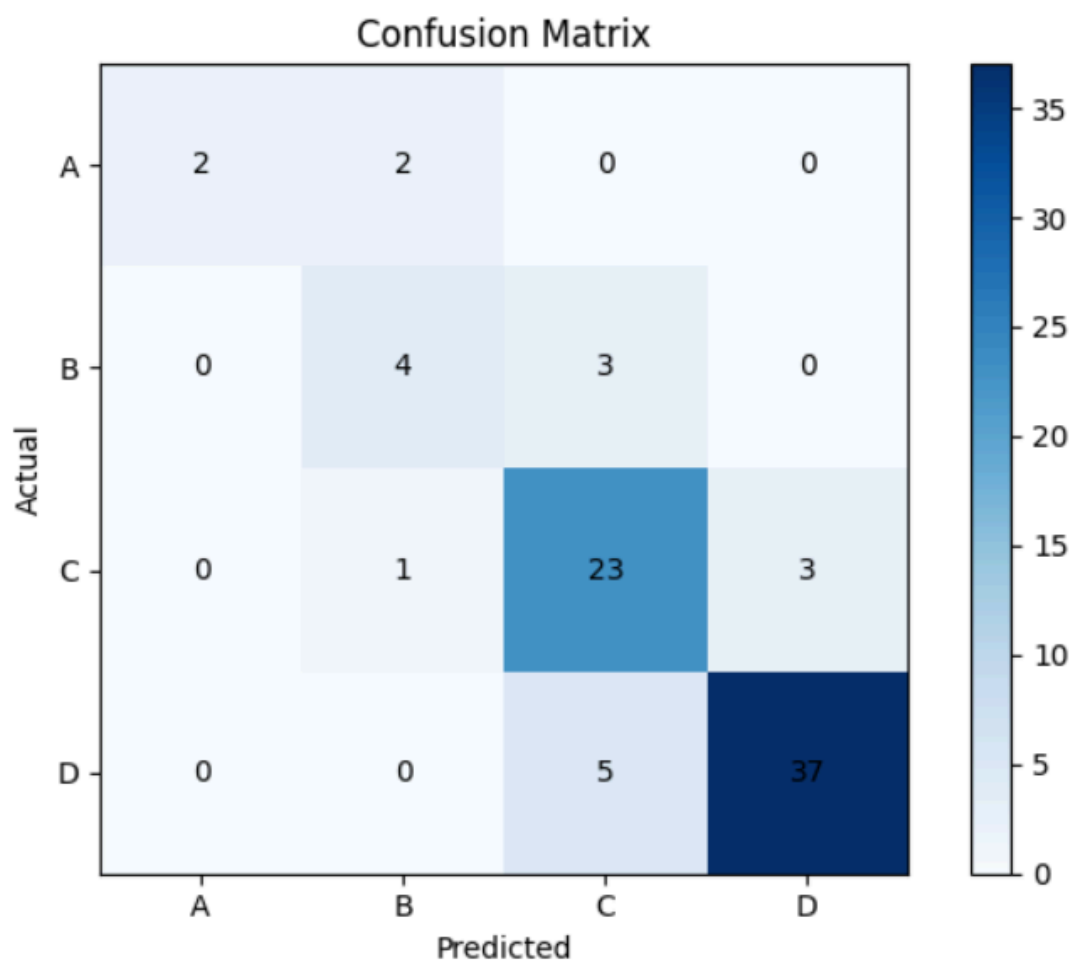
Max_depth = برای جلوگیری از ریز شدن و دقت زیاد روی **train data** نباید اینو زیاد بذاریم .

min_sample_split = هرچه ویژگی هایی که باید در یک گره باشند بیشتر باشه مدل تصمیم های متنوع تری میتونه بگیره و به جزییات بیشتر توجه میکنه که باعث **overfitting** میشود .

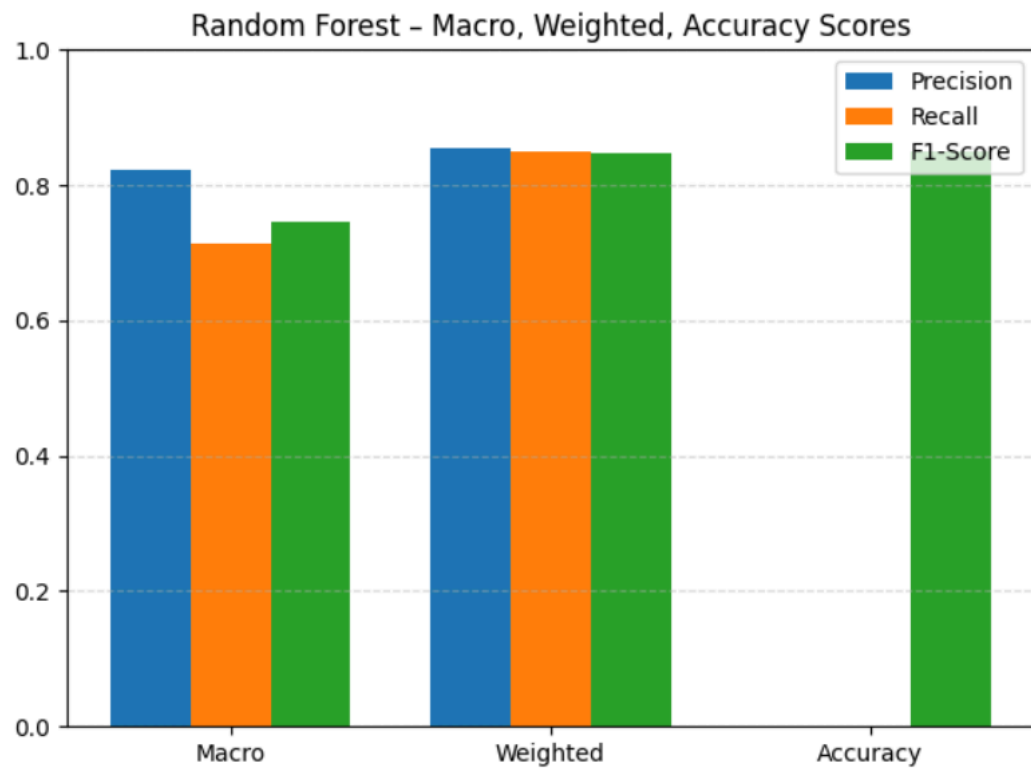
min_sample_leaf = هرچه تعداد نمونه هایی که باید در برگ باشند بیشتر باشد مدل روی جزییات بیشتر حساس شده و دقت بالاتری روی **train data** پیدا میکنه که موجب **overfitting** میشه و هرچه بیشتر باشه نمونه ها درخت ساده تر و بهتر عمل میکنه .

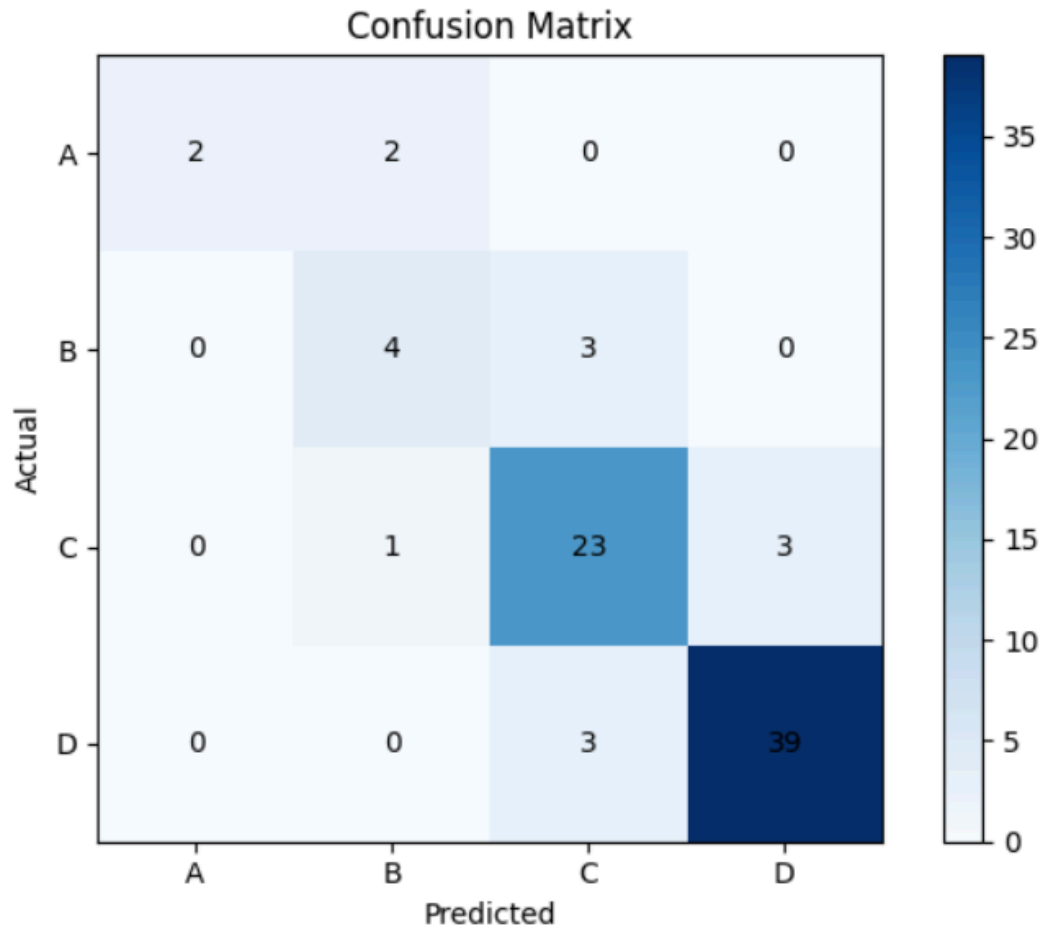
Feature_importance = ویژگی هایی که اهمیت بالاتری دارند باید در ارتفاع بالاتری باشند تا مدل سریع تر بتواند کلاس بندی بکند که با توجه به شکل **dsgrade** از اهمیت بالاتری برخوردار هست و تاثیر بیشتری میگذارد روی مدل پس ان را به عنوان گره اول انتخاب میکنیم .





[Random Forest](#)

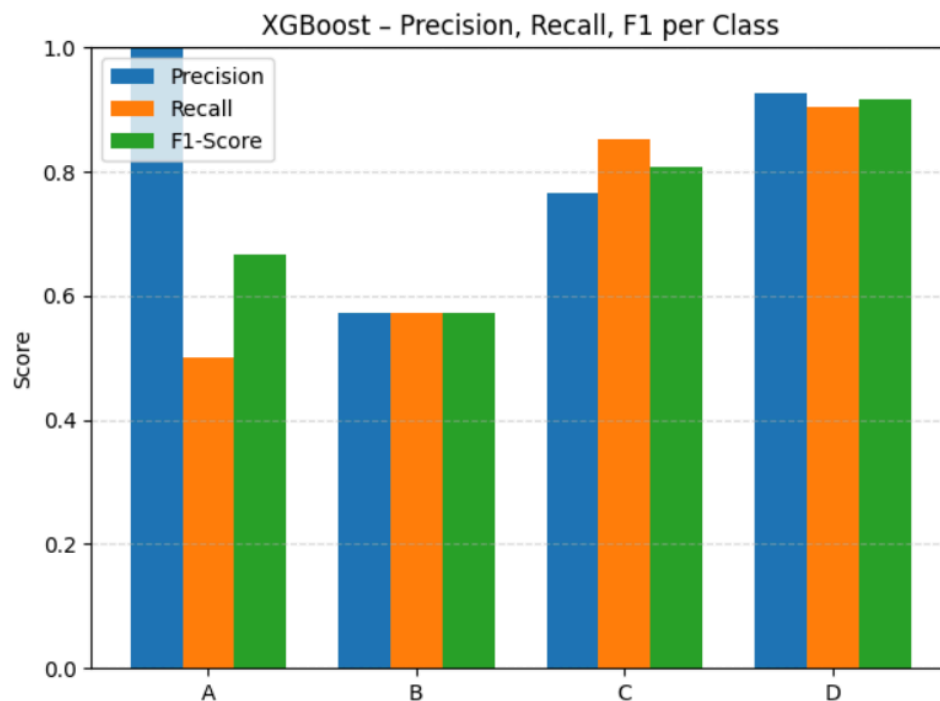


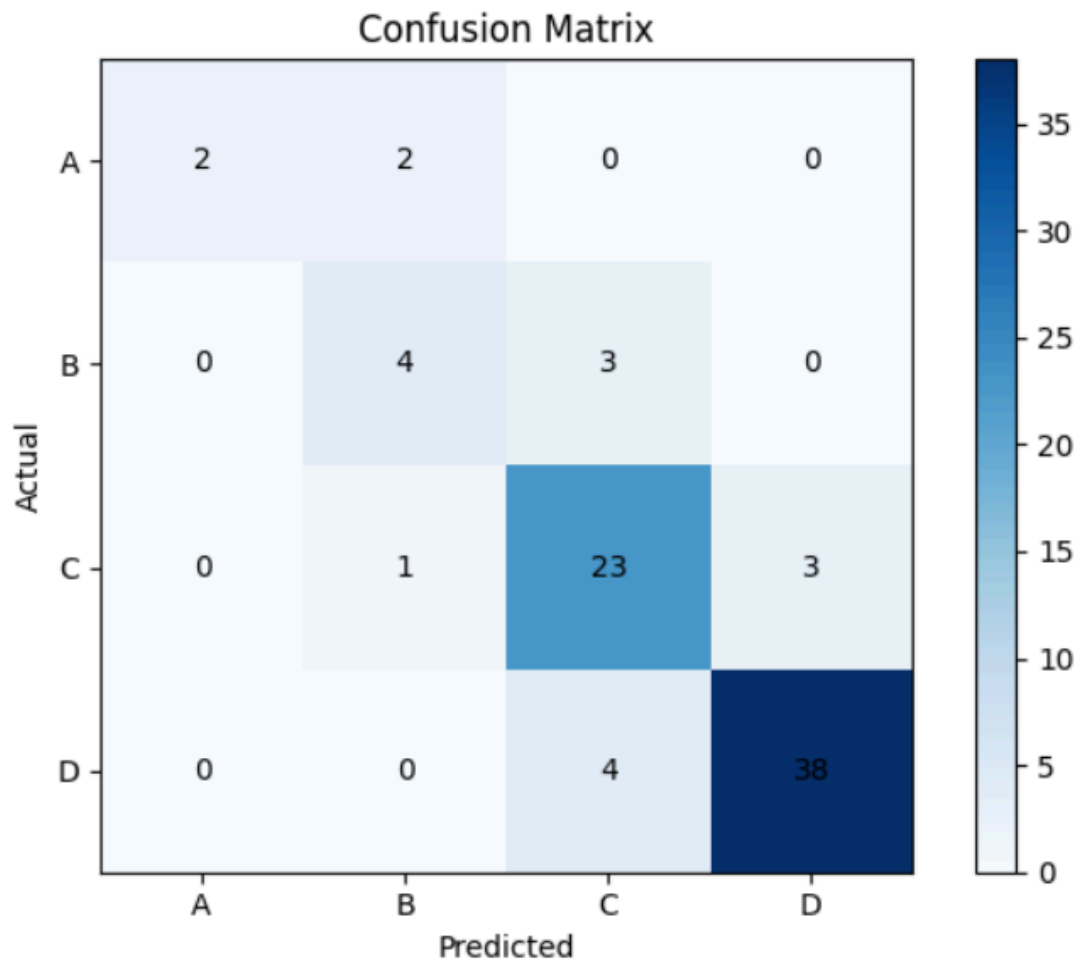


XGBoost

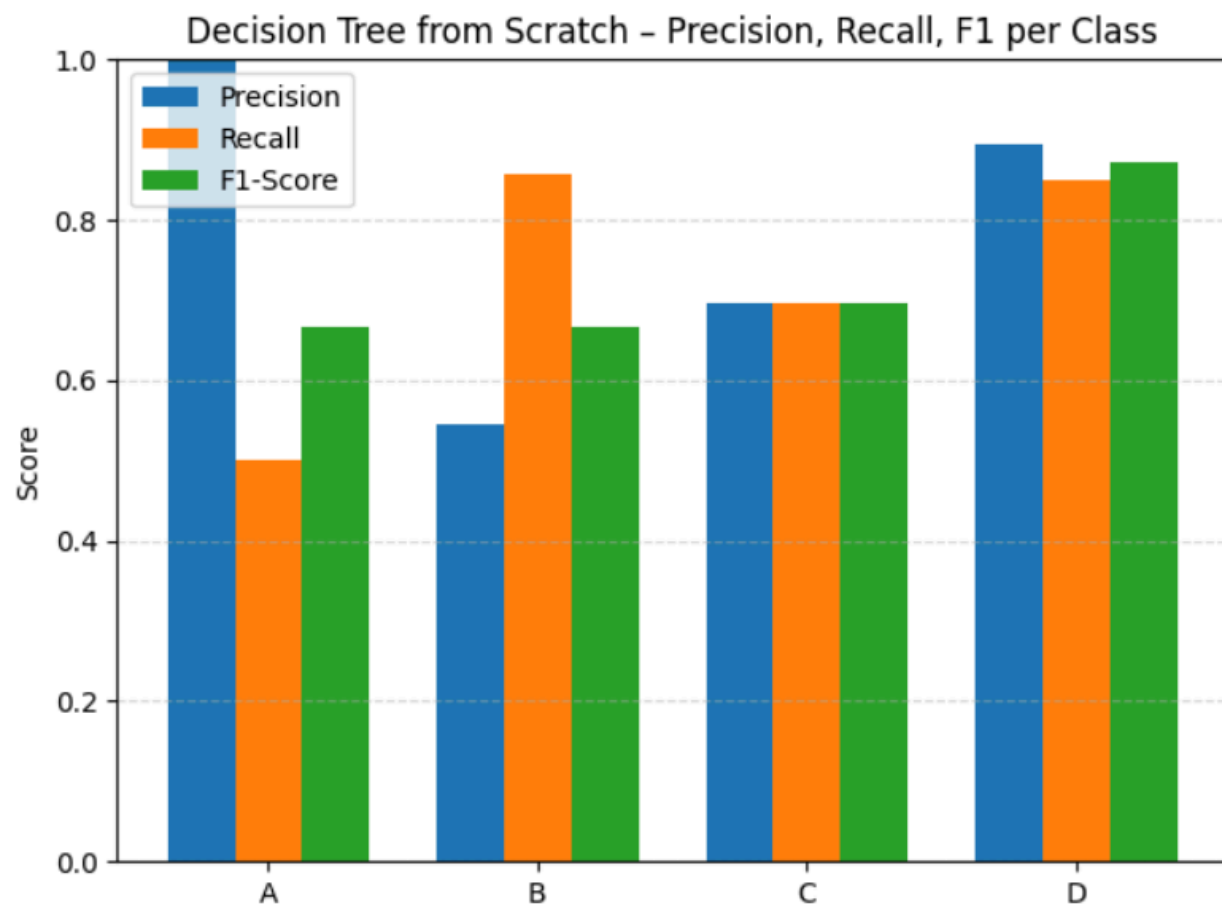
- **learning rate:** نشان می‌دهد که مدل در هر مرحله تقویتی (boosting) چقدر وزن تغییر بدهد. هرچه عدد بالاتر باشد سرعت یادگیری بالاتر و دقت پایین‌تر هست که در این مثال 0.01 تعادل خوبی بین آن‌ها بوده.
- **n estimates:** تعداد درخت‌هایی که مدل برای یادگیری می‌سازد. تعداد کمتر سریع‌تر ولی ممکنه خوب یادگیره. مقدار نهایی آن باید با learning rate هماهنگ باشد.
- **min_samples_split:** اجازه تقسیم شدن در نودها که هرچی کمتر باشد راحت‌تر در نودها تقسیم میشوند و باعث overfitting احتمال دارد بشوند.

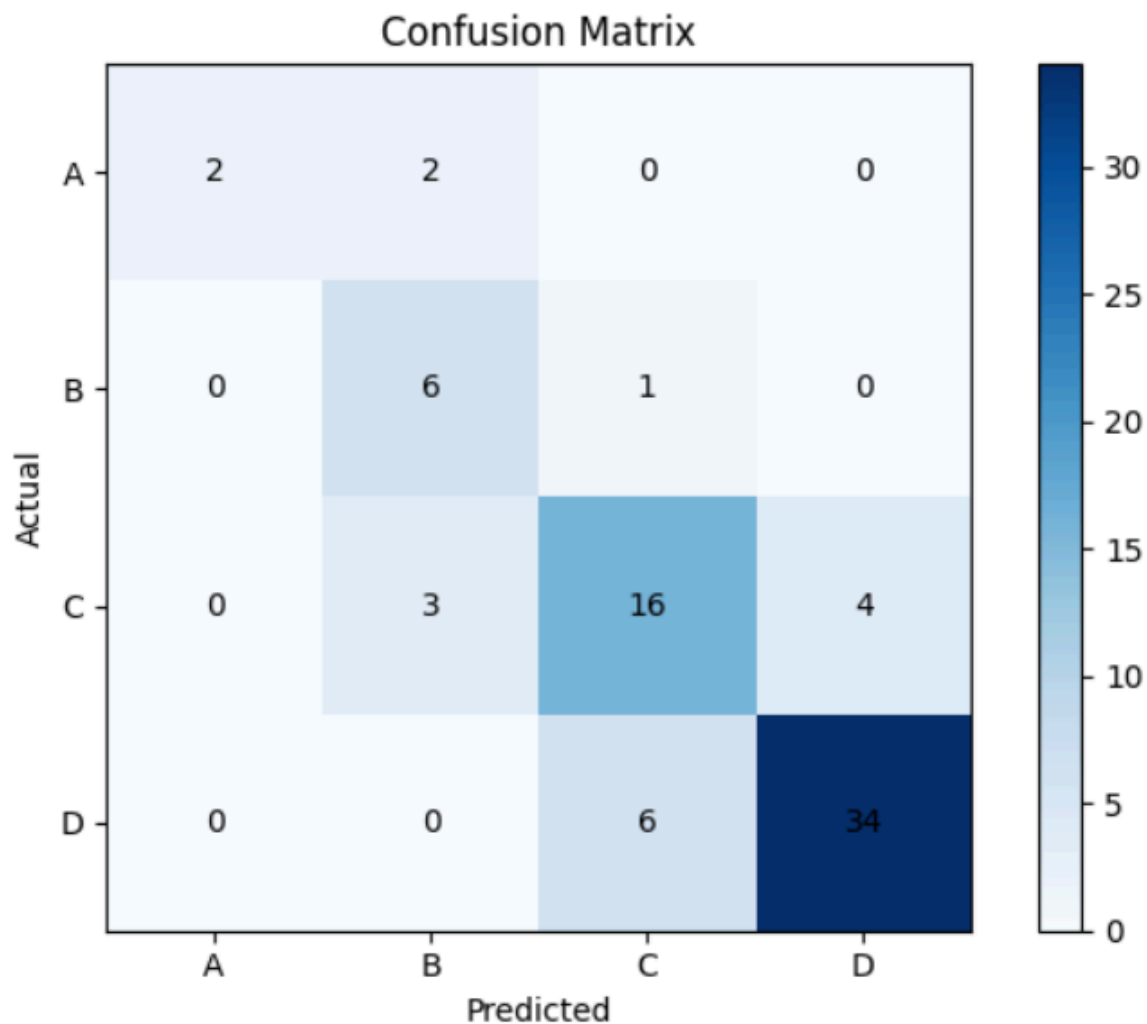
- `min_samples_leaf`: حداقل نمونه در برگ ها که اگر خیلی کم باشد باعث `overfitting` میشود .
- `max_depth`: عمق بیشتر دقت بیشتر و خطای `overfitt`
- `max_features`: هرچقدر ویژگی ها بیشتر باشه واریانس کمتر میشود .





[Decision Tree from Scratch](#)





[Comparison with Library Implementation](#)

در مدلی که خودم نوشتم $\text{accuracy } 0.78$ و در مدل آماده 0.81 هست که یعنی دقت بالاتری دارد. از دلایل آن قدرت بالاترش در هرس کردن درخت و کار کردن با داده های پیوسته هست.